

Vorwort

Im Wintersemester 17/18 hat eine Gruppe von Studenten bereits eine mobile Ladebox für Elektroautos entwickelt und umgesetzt. Aus zeitlichen Gründen kam es allerdings nicht zum

vollständigen Abschluss des Projekts. Die Ladefunktion konnte nicht hergestellt werden.

Unsere Aufgabe ist es an der gemachten Arbeit anzuknüpfen und die Ladefunktion inklusive Erweiterungen herzustellen.

Neben der funktionierenden Ladebox wollen wir einen Emulator entwickeln der ein zu ladendes Auto simuliert.

Außerdem soll die Freischaltung der Station über eine RFID Karte funktionieren. Ein Display soll den jeweiligen Zustand der Ladebox anzeigen („Betriebsbereit“, „Laden“, „Störung“, etc.).

Über eine Schnittstelle soll eine Verbindung zu einem Webserver aufgebaut werden können.

Inhaltsverzeichnis

1. Planung

1.1 MoSCoW-Prinzip

1.2 Vorgehensweise

2. Hardware Ladebox

2.1 Analyse und Fehlersuche

2.2 Das Statussystem

2.3 Inbetriebnahme

2.4 Funktion/Aufbau der Ladebox

2.5 Hauptstromkreis

2.6 Steuerstromkreis

3. Der Emulator

3.1 Funktionsbeschreibung

4. Software Raspberry PI

4.1 Komponenten

4.1.1 LCD-Display

4.1.2 RFID Modul RC522

4.1.3 EV Charge Control

4.2 Ladesteuerung

4.3 Webserver

5. Fazit

1. Planung

1.1 MoSCoW-Prinzip

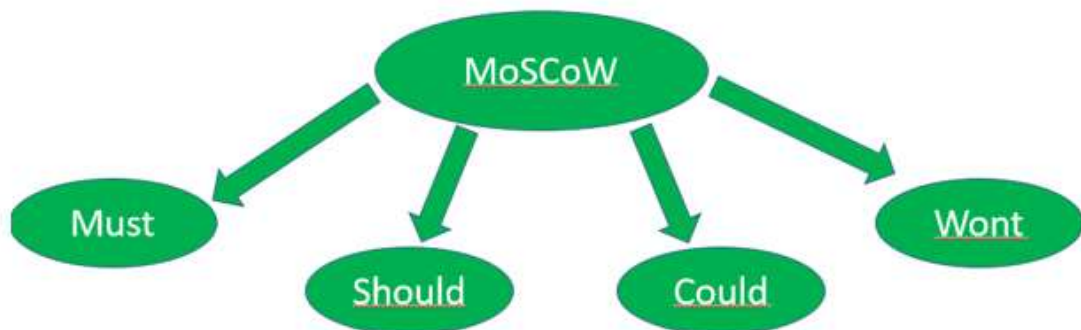
In einer ersten Planungsphase machten wir eine Bestandsaufnahme und erstellten nach Absprache mit unseren Betreuern eine Prioritätenliste nach dem MoSCoW-Prinzip auf.

M – MUST: unbedingt erforderlich

S – SHOULD: sollte umgesetzt werden, wenn alle MUST-Anforderungen trotzdem erfüllt werden können

C – COULD: kann umgesetzt werden, wenn die Erfüllung von höherwertigen Anforderungen nicht beeinträchtigt wird

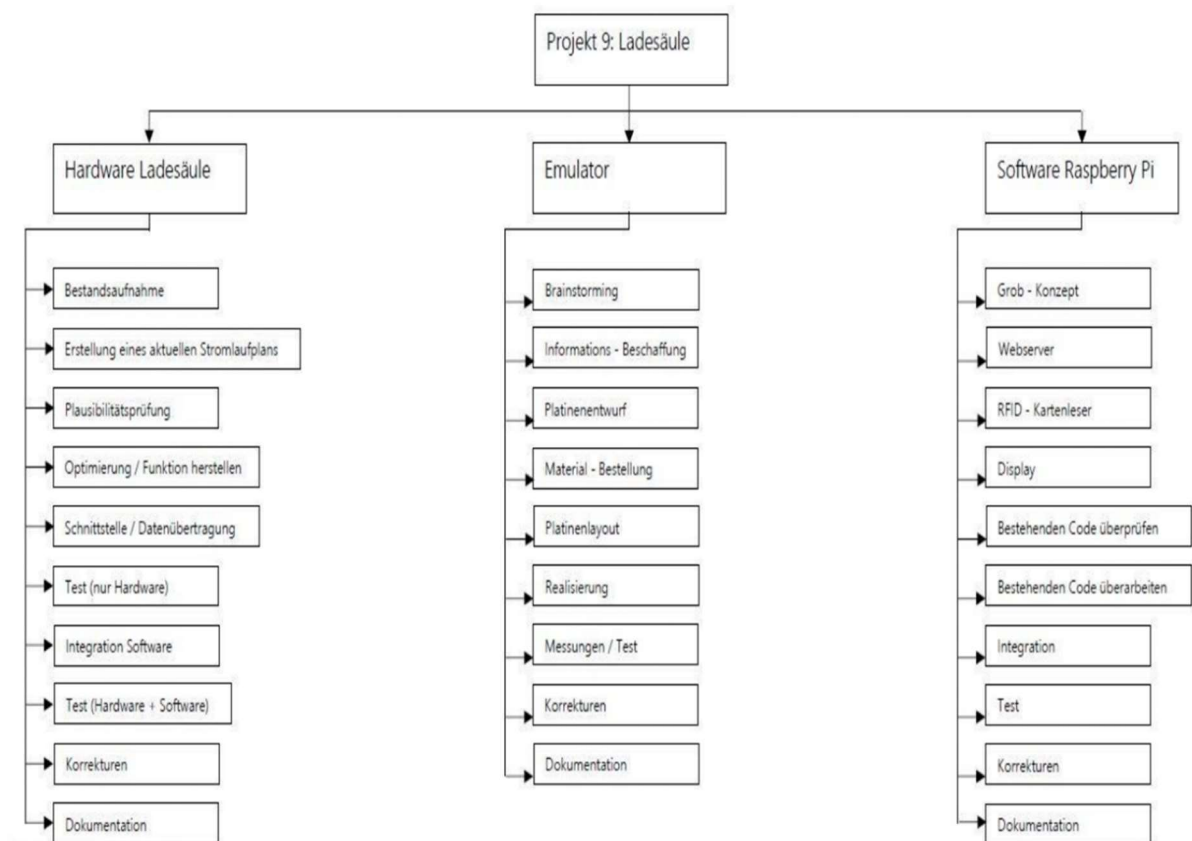
W – WON'T: wird diesmal nicht umgesetzt, aber für die Zukunft vorgemerkt



- Ladefunktion herstellen
- Freigabe der Ladebox und Authentifizierung des Nutzers über RFID-Kartenleser
- Anzeigen des Status der Ladebox
- Abrechnungsmöglichkeit durch Auslesen des Stromverbrauchs über den Zähler
- Emulator: Simulation unterschiedlicher Fahrzeuge
- Rückmeldung, ob Ladebox richtig arbeitet
- Erweiterung des Zugangs auf mehrere Nutzer
- Anzeige über LCD-Display
- Zugriff auf Ladebox über Web-Server und Statusabfrage
- Ladeleistungsbegrenzung bei mehreren Ladeboxen
- Emulator: Anzeige über LCD-Display und Analyse der PWM

1.2 Vorgehensweise

Nach Aufstellung unseres Konzepts legten wir Zuständig- und Verantwortlichkeiten fest, um ein paralleles Abarbeiten der Teilaufgaben zu ermöglichen. Im Groben unterteilten wir unser Projekt in drei Hauptgebiete: Hardware Ladesäule, Emulator, Software Raspberry Pi. Die folgende Grafik (Grafik 1.2.1) zeigt die einzelnen Vorgänge unseres Projekts in ihrer zeitlichen Abfolge.



Grafik 1.2.1 Projektplan

2. Hardware Ladebox

2.1 Analyse und Fehlersuche

Die Ladesäule mit all ihren Komponenten haben wir von der Vorgängergruppe übernommen. Da die Funktion an sich, d.h. das reine elektrische Laden eines Autos ohne jeglicher Software noch nicht gegeben war, galt es diese als erstes sicherzustellen. Nach ausgiebiger Schaltplananalyse und Internetrecherche konnten die Verdrahtungsfehler erkannt und behoben werden und ein fehlerhaftes Bauteil ausgetauscht werden. Dabei handelte es sich um den Verriegelungsmotor der Mennekes Ladebuchse, das vorhandene 24 V Bauteil musste gegen ein 12 V Motor ausgetauscht werden.

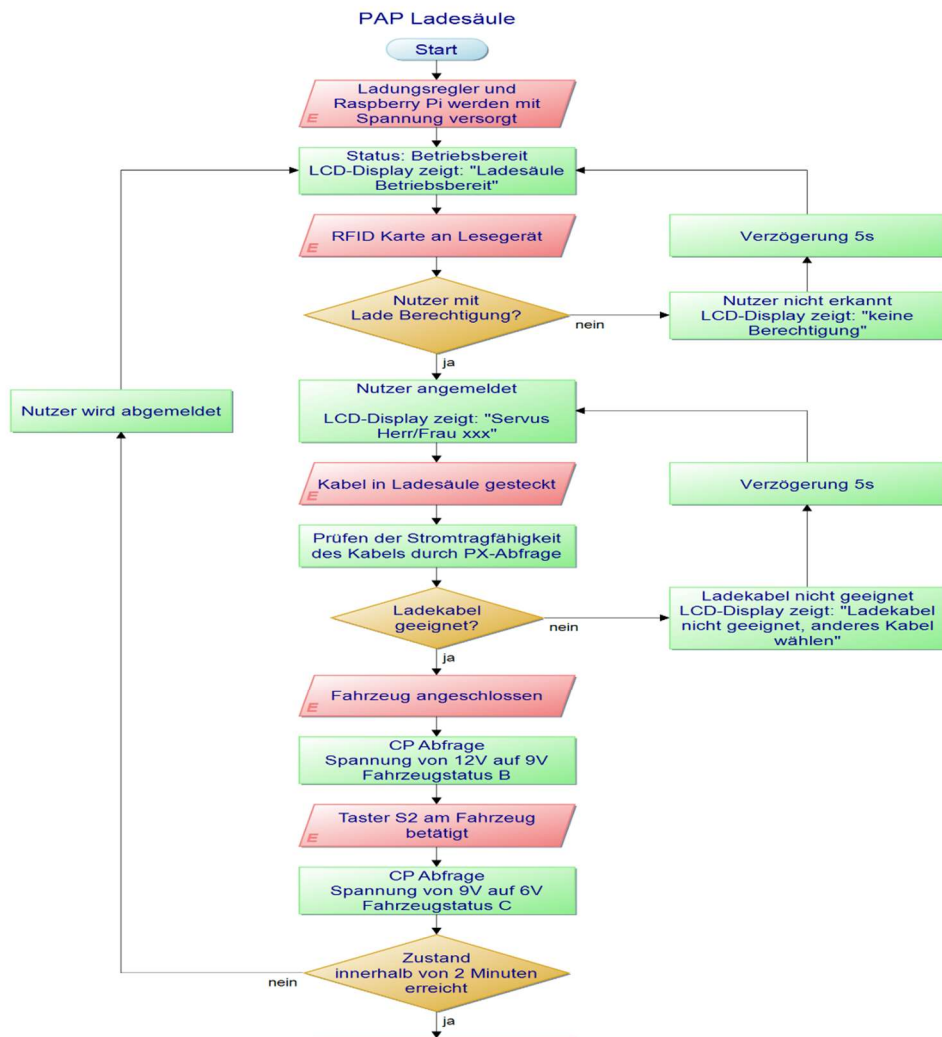
2.2 Das Statussystem

Fahrzeug-status	Fahrzeug ange-schlossen	S2 ¹	Ladevorgang möglich	Va ²	Beschreibung
A	Nein	Offen	Nein	12 V	Vb ³ = 0 V
B	Ja	Offen	Nein	9 V	R2 erkannt B1 (9 V DC): EVSE ⁴ noch nicht bereit B2 (9 V PWM): EVSE bereit
C	Ja	Geschlossen	Fahrzeug bereit	6 V	R3 = 1,3 kΩ ±3 % Belüftung nicht erforderlich
D				3 V	R3 = 270 Ω ±3 % Belüftung erforderlich
E	Ja	Offen	Nein	0 V	Vb = 0: EVSE, Kurzschluss am EV Charge Control, Spannungsversorgung nicht verfügbar
F	Ja	Offen	Nein	EVSE nicht verfügbar	EVSE nicht verfügbar

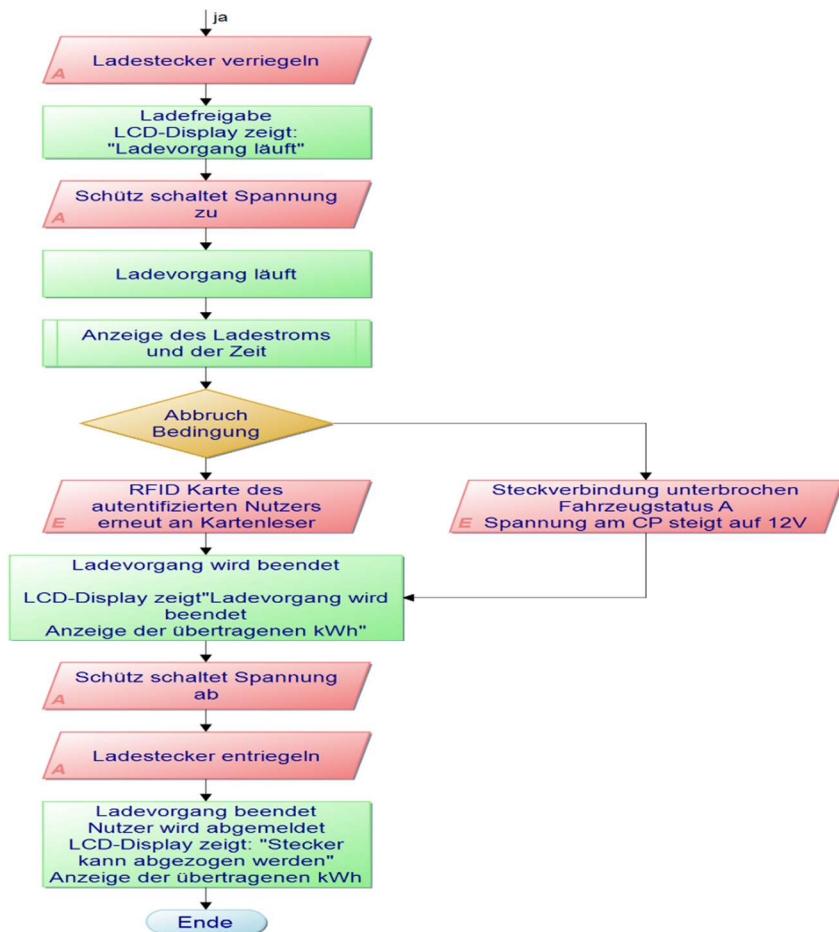
Grafik 2.2.1 Tabelle Statussystem

2.3 Inbetriebnahme

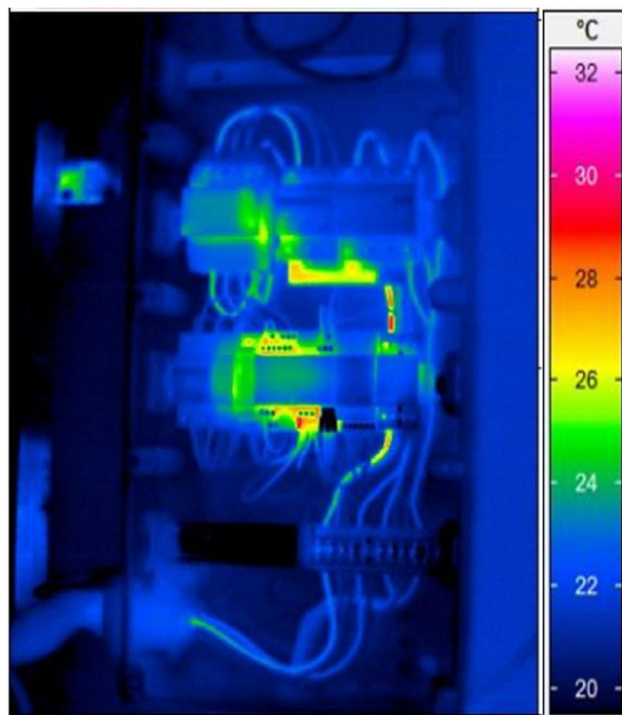
Den ersten Test nahmen wir an unserem Emulator vor und da dieser erfolgreich war, konnten wir einen BMW i3 an unsere Ladebox anschließen. Das Laden des Fahrzeugs funktionierte ebenfalls einwandfrei. Das Inbetriebnahmeprotokoll ist der Grafik 2.3.1 zu entnehmen. Zudem ist ein Wärmebild der Ladebox bei einphasigen Betrieb zu sehen (Grafik 2.3.2). Phase L1 zeigt einen deutlichen farblichen Unterschied.



Grafik 2.3.1 Inbetriebnahmeprotokoll



Grafik 2.3.1 Inbetriebnahmeprotokoll Fortsetzung



Grafik 2.3.2 Wärmebild der Ladebox bei einphasiger Ladung

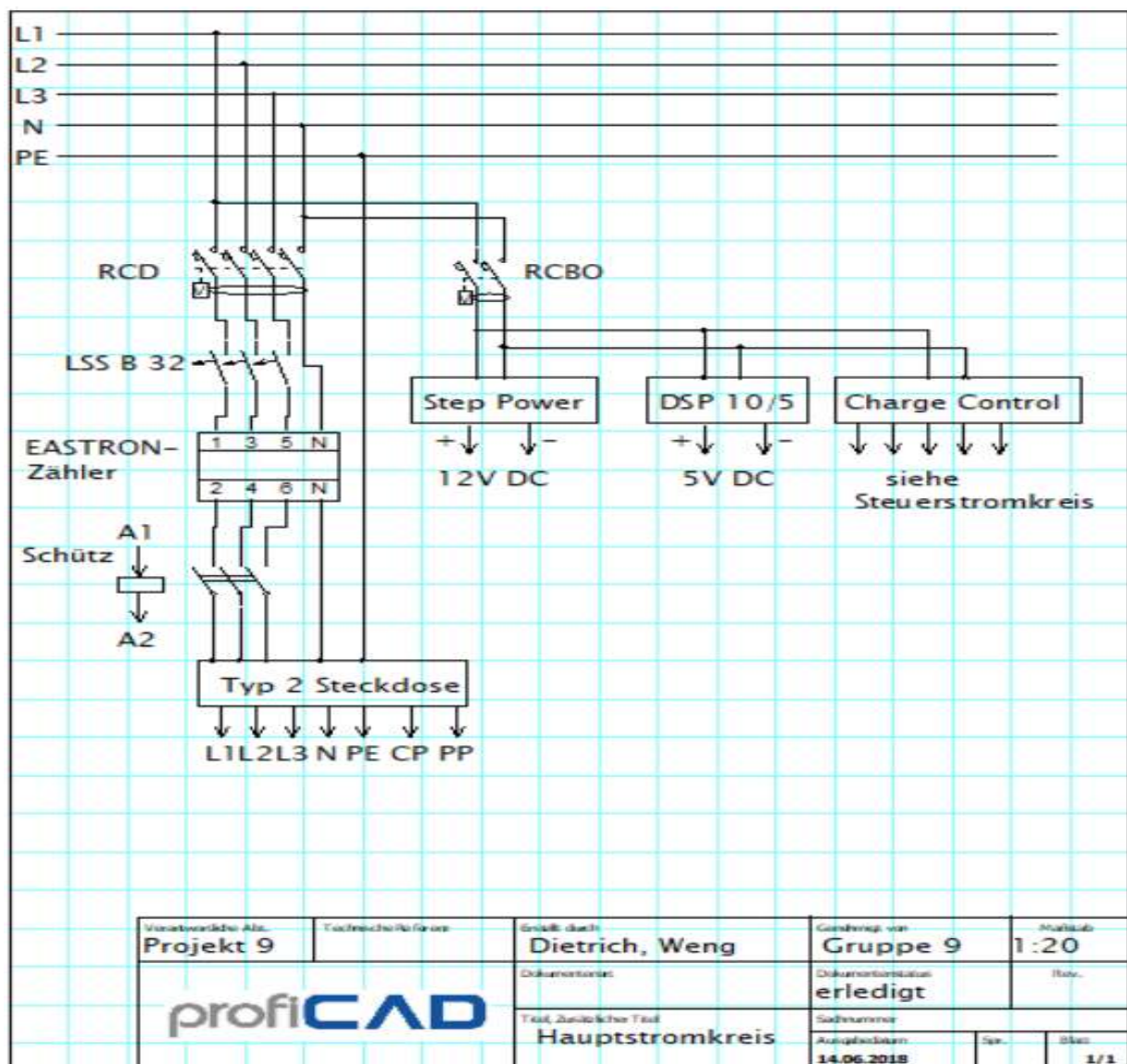
2.4 Funktion/Aufbau der Ladebox

Die Ladebox besteht aus einem Hauptstromkreis und einem Steuerstromkreis. Der Laststromkreis ist über einen FI-Schutzschalter (ABB) und einen Leitungsschutzschalter (ABB) gegen Kurzschluss- und Fehlerströme geschützt. Außerdem beinhaltet er einen Energiezähler (EASTRON) und einen Leistungsschutz (ABB) der die Last, in diesem Fall, die Ladebuchse zu schaltet.

Der Steuerstromkreis ist ebenfalls gegen Kurz- und Fehlerströme über eine Kombination aus FI-Schutzschalter und Leitungsschutzschalter (ABB) geschützt. Als Herzstück besitzt er den EV-Charge Control (Phoenix Contact). Dieser regelt den vollständigen Ladevorgang und ist gleichzeitig Schnittstelle zum Raspberry Pi. Versorgt wird dieser mit 12 V von einem Netzteil Step Power (Phoenix Contact). Das Bauteil EV Charge (Phoenix Contact) ist für die Notentriegelung zuständig. Im Fehlerfall z.B. Stromausfall entriegelt dieser noch und gibt so den Stecker frei.

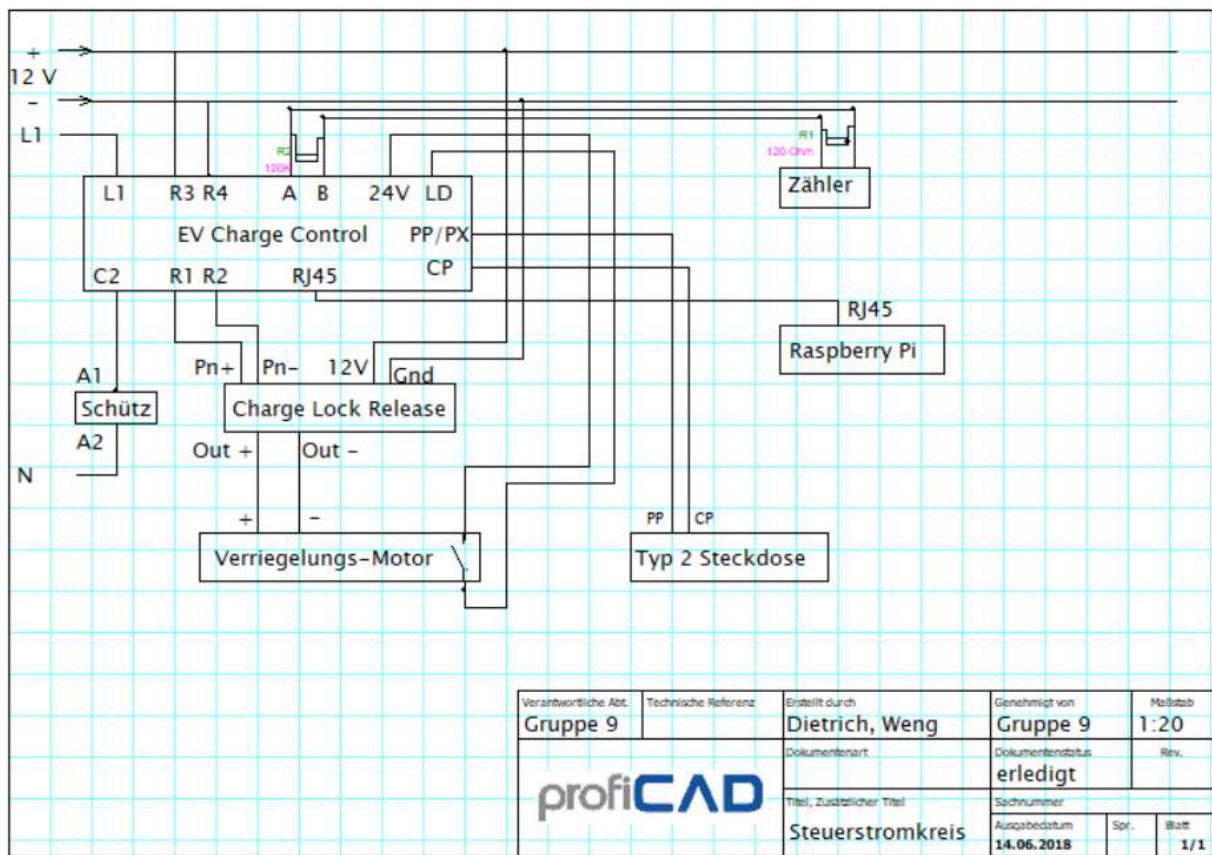
Die exakte Verdrahtung zeigen die beiden folgenden Schaltpläne:

2.5 Hauptstromkreis



Grafik 2.5.1 Hauptstromkreis

2.6 Steuerstromkreis



Grafik 2.6.1 Steuerstromkreis

3. Der Emulator

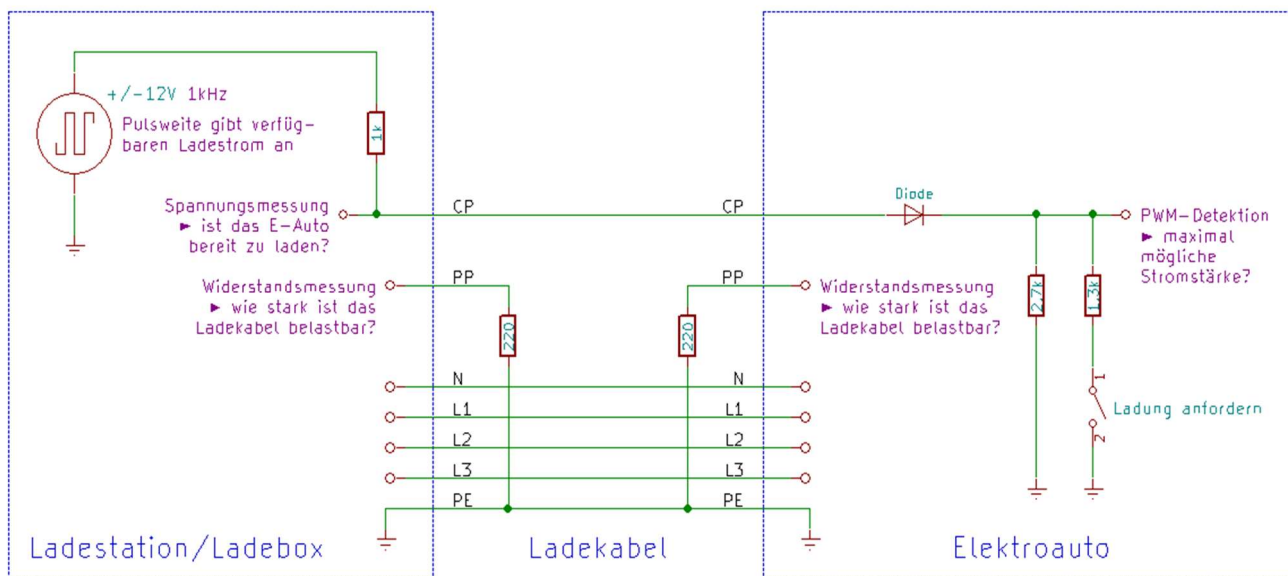
Während es bei einem normalen Haushaltsgerät genügt den Stecker in die Steckdose zu stecken und das Ladegerät sich selbst den benötigten Strom zieht, ist bei einem Elektroauto durch die teilweise sehr hohen Ladeströme die Technik etwas komplizierter. Um Gefahren für Mensch und Technik zu vermeiden muss ein Ladevorgang kontrolliert gestartet, durchgeführt und beendet werden. Hierfür gibt es ein Status-System, welches die Kommunikation zwischen Ladesäule und Fahrzeug darstellt (genauer unter 2.2 Das Statussystem).

3.1 Funktionsbeschreibung

Da zahlreiche Tests der Hardware und der Software erforderlich waren erschien es insofern als sinnvoll, einen Emulator zu entwickeln. Dieser sollte es ermöglichen ohne einen größeren Aufwand die Funktionen der Ladesäule im Labor zu testen. Des Weiteren war ein wichtiger Grund für die Entwicklung, dass damit die komplexe Technik eines E-Fahrzeuges im Fehlerfall nicht beschädigt wird. Ein weiterer Vorteil stellt die Tatsache dar, dass mehrere unterschiedliche Fahrzeuge simuliert werden können und nicht nur stichprobenartig ein E-Auto.

Technisch gesehen waren die Funktionen, die der Emulator liefern sollte, dass man mit Hilfe von Kontrollleuchten sehen kann, ob die drei Phasen L1 bis L3 funktionieren. Unter anderem lässt sich erkennen, ob das Fahrzeug ein oder dreiphasig lädt oder im Fehlerfall eine Phase ausgefallen ist.

Eine weitere Funktion ist, mit Hilfe eines Schalters verschiedene maximal mögliche Ladesströme des Fahrzeugs zu simulieren. Diese Information wird mit Hilfe des CP-Kontakts (control pilot) übermittelt. Die Charge Control in der Ladebox stellt für den CP 12 V zur Verfügung. Ist kein Fahrzeug angeschlossen befindet sich der Kontakt im Leerlauf und es werden entsprechend 12 V gemessen. Sobald das Fahrzeug angesteckt wird stellt sich ein Spannungsteiler ein, wodurch die Charge Control noch 9 V misst und damit feststellt, dass ein Fahrzeug angeschlossen ist und entsprechend die Verriegelung aktiviert. Mit Hilfe eines Schalters am Emulator lässt sich zwischen einem Widerstand mit 1,3 kOhm und einem mit 270 Ohm wählen. Entsprechend fällt die Spannung auf 6V bzw 3V ab und die Charge Control weiß wie hoch die maximale Stromverträglichkeit des Fahrzeugs ist. Wenn 0 V gemessen wird besteht ein Kurzschluss am EV Charge Control, was zu einem Fehler führt. Die entsprechende Schaltung ist der folgenden Grafik 3.1.1 zu entnehmen.



Grafik 3.1.1 Kommunikationsprotokoll

Des Weiteren kann der CP-Kontakt mittels Pulsweitenmodulation dem Fahrzeug mitteilen wie hoch die maximale Stromstärke ist, die die Ladesäule zur Verfügung stellen kann. Dies ist in einem Bereich von 6A – 80A möglich. Das genaue Verhältnis kann der nachfolgenden Tabelle 3.2 entnommen werden. Zur Realisierung war geplant mit Hilfe eines Raspberry im Emulator das Tastverhältnis auszumessen und mit Hilfe eines Displays an der Oberfläche den Wert auszugeben. Eine Umsetzung war aus Zeitgründen nicht mehr möglich, ist aber z.B. im Rahmen eines Nachfolgeprojekts noch nachrüstbar.

Auswertung der Nenntastverhältnisse durch das Fahrzeug	Maximaler Strom, der vom Fahrzeug entnommen werden darf
Tastverhältnis < 3 %	Ladevorgang ist nicht erlaubt
$3 \% \leq \text{Tastverhältnis} \leq 7 \%$	Zeigt an, dass die digitale Kommunikation benutzt wird, um ein Gleichspannungsladegerät außerhalb des Fahrzeugs zu steuern oder um den verfügbaren Stromwert an ein Ladegerät innerhalb des Fahrzeugs zu übermitteln. Die digitale Kommunikation kann auch mit anderen Tastverhältnissen verwendet werden. Der Ladevorgang ist nur mit digitaler Kommunikation erlaubt. 5 % Tastverhältnis sollte benutzt werden, wenn die Pilot-Leitung für die digitale Kommunikation benutzt wird.
$7 \% < \text{Tastverhältnis} < 8 \%$	Ladevorgang ist nicht erlaubt
$8 \% \leq \text{Tastverhältnis} < 10 \%$	6 A
$10 \% \leq \text{Tastverhältnis} \leq 85 \%$	Verfügbarer Strom = (% Tastverhältnis) x 0,6 A
$85 \% < \text{Tastverhältnis} \leq 96 \%$	Verfügbarer Strom = (% Tastverhältnis - 64) x 2,5 A
$96 \% < \text{Tastverhältnis} \leq 97 \%$	80 A
Tastverhältnis > 97 %	Ladevorgang ist nicht erlaubt

Grafik 3.1.2 Tabelle-Tastverhältnis

Der PP-Kontakt (proximity plug), der mit Hilfe eines Widerstandes im Kabel die maximale Strombelastbarkeit dieses angibt könnte über eine entsprechende Widerstandsschaltung simuliert werden. Die Idee dies zu realisieren wurde verworfen, da sich die Möglichkeit ergab direkt verschiedene Kabel zu verwenden und dadurch diese gleich mit zu testen.

Um den Emulator zu realisieren, galt es vorerst ein passendes Gehäuse zu finden, welches vor allem genug Platz für die Ladedose (Typ 2) bietet. Geeignete Verteilerdosen oder kleinere Unterverteilungen waren meist ungeeignet oder sehr teuer. Um dem Aussehen eines Elektroautos gerecht zu werden, kamen wir letztendlich auf die Idee ein Bobby Car zu verwenden. Hierbei waren die Schwierigkeiten die Bauteile optisch ansprechend zu positionieren. Die Tatsache, zum einen das Innenleben, wie die Platine, jederzeit zugänglich zu machen, zum anderen aber auch für den Endbenutzer berührungssicher und möglichst unsichtbar zu gestalten, führte zu einer verschließbaren Klappe am Unterboden des Fahrzeugs. Die Platine wurde an der Innenseite der Klappe fest verschraubt. Um den Deckel im Betrieb fest zu verschließen wurden zwei Flügelschrauben verwendet. Die Ladedose und Kontrollleuchten wurden an der Oberseite montiert, der Drehwahlschalter für den CP-Kontakt an der rechten Seite. Die Signalpins der Dose mussten verlötet werden, die restlichen Kontakte konnten mit Schraubklemmen hergestellt werden. Im Inneren des Bobby Cars ist nach wie vor genug Platz für mögliche Erweiterungen. Insgesamt wirkt der Emulator so optisch viel ansprechender und kommt dem Grundgedanken ein Elektrofahrzeug darzustellen wesentlicher näher. Hinzuzufügen ist, dass ein Bobby Car preislich günstiger ist als sämtliche elektrotechnische Gehäuse.

4. Software Raspberry Pi

4.1 Komponenten

4.1.1 LCD-Display

Über I2C ist das LCD-Display mit dem Raspberry verbunden. I2C des Raspberry muss zuerst freigeschaltet werden. Für die Kommunikation mit dem LCD-Display wird die Library `lcdlib` verwendet, die man unter <http://www.netzmafia.de/skripten/hardware/RasPi/Projekt-LCD/lcdlib.py> finden kann. Der LCD-Display besitzt 4 Zeilen mit jeweils maximal 15 Zeichen pro Zeile. Die Adresse des Display wird über die Variable `ADRESS` und der Funktion `lcdlib.lcd(ADRESS, 4, 16)` auf `0x3f` gesetzt.

Mit der Funktion `lcd.set_cursor(Positionzeichen, Positionzeile)` wird der Cursor gesetzt. Ein String kann dann über die Funktion `lcd.write_string(STRING)` ausgegeben werden.

4.1.2 RFID Modul RC522

Über RFID Karten soll die Freigabe des Ladevorgangs erteilt werden. Diese werden überein Raspberry Pi RFID Modul RC522 ausgelesen, welches über SPI angebunden ist. Die SPI Schnittstelle des Raspberry Pi muss ebenfalls aktiviert werden. Für die Anwendung des Moduls wird die Library `MFRC522` verwendet, welche unter <https://github.com/mxgxw/MFRC522-python.git> zu finden ist. Mit Hilfe von `MFRC522.MFRC522()` wird ein neues `MIFAREReader` Objekt erzeugt. Jede RFID Karte besitzt eine UID (Unique Identification) Code, welcher nicht veränderbar ist und ein Key der jederzeit umgeschrieben werden kann.

Durch `(status, TagType) = MIFAREReader.MFRC522_Request(MIFAREReader.PICC_REQIDL)` wird nach Karten gescannt. Falls eine Karte erkannt wird, wird die UID der Karte ausgelesen mit `(status, uid) = MIFAREReader.MFRC522_Anticoll()`. Anschließend kann mit `status = MIFAREReader.MFRC522_Auth(MIFAREReader.PICC_AUTHENT1A, 8, key, uid)` überprüft werden ob der Key der Karte mit dem in der Variable `key` übereinstimmt. Alle vorhandenen RFID Karten wurden auf den Default Key `[0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF]` gelassen. Im Code wird nach genau diesen Key gesucht. Stimmen die Keys überein dann wird `status` auf `True` gesetzt.

4.1.3 EV Charge Control

Die Raspberry Pi ist über Ethernet an den EV Charge Control angeschlossen und kommuniziert mittels Modbus/TCP. Für diese wird die Python pyModbusTCP Library verwendet. Diese ist unter <https://github.com/sourceperl/pyModbusTCP/tree/master/pyModbusTCP> zu finden. Zum Verbindungsaufbau werden 3 Variablen definiert `SERVER_HOST` die die IP Adresse "10.27.210.76" des EV Charge Control enthält, `SERVER_PORT` beträgt 502 und `UNIT_ID` 180. Anschließend wird eine Variable für den Modbus Client angelegt über `ChargeControl = ModbusClient()`. Mithilfe von `ChargeControl.host(SERVER_HOST)`, `ChargeControl.port(SERVER_PORT)` und `ChargeControl.unit_id(UNIT_ID)` werden dem Client die essentiellen Parameter übergeben um eine Verbindung zu ermöglichen. Die Verbindung kann dann über `ChargeControl.open()` geöffnet werden.

Das Auslesen von EV-Status, Ladefreigabe und Verbrauch erfolgt über verschiedene Funktionen abhängig vom Registertypen, in dem sich der Werte befindet. Für den EV-Status wird die Funktion: `ChargeControl.read_input_registers(100,1)` verwendet die zwei Funktionsargumente stehen für Adresse, und Anzahl an auszulesenden Register. Hingegen wird für das Auslesen des Freigabebit die Funktion `read_coil(0x190,1)` verwendet die Funktionsargumente haben die gleiche Bedeutung wie schon beim EV-Status.

Beim Auslesen von Messwerten über den EV Charge Control des Messgerätes treten noch einige Fehler auf. Der EV Charge Control hat eine Weboberfläche, in der man Einstellungen vornehmen kann unter welchen Adressen, welche Messwerte des Messgeräts zu finden sind. Theoretisch würde über `ChargeControl.read_holding_registers(341,1)` man den Verbrauch als Rückgabewert erhalten allerdings wird lediglich die Adresse zurückgegeben, die im Webserver eingetragen ist. Bei direkter Kommunikation zwischen Rechner und Messgerät über ein Ethernet Gateway und unter Verwendung der im Datenblatt des Messgeräts festgehaltenen Registernummern erhält man keine bzw. "0" zurück. Beim Versuch der IP-Adressen Änderung des Ethernet Gateways findet man zwar die IP-Adresse in den Registern allerdings nicht an der angegebenen Stelle.

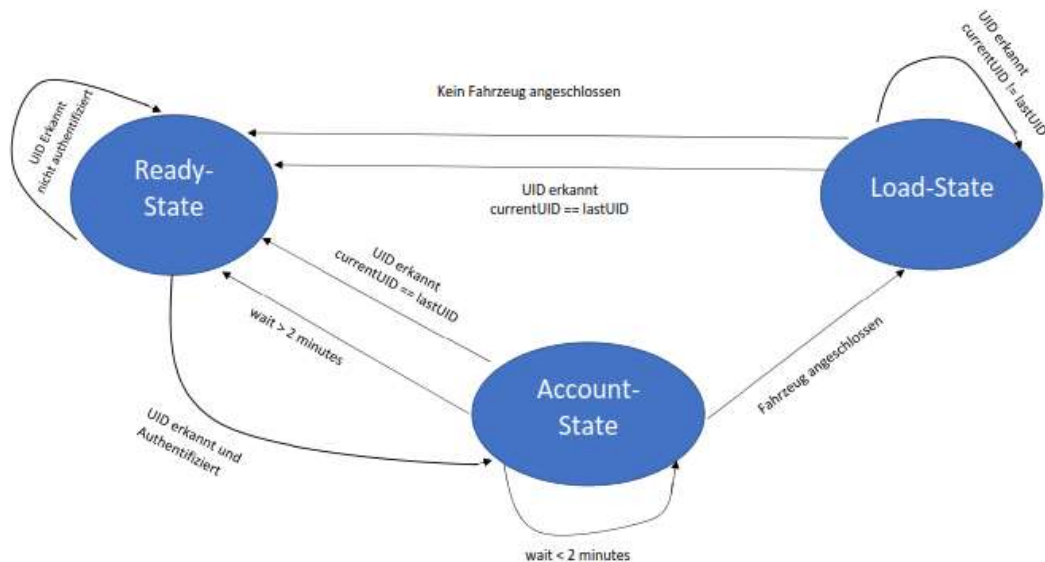
4.2 Ladesteuerung

Die Ladesteuerung ist in Python realisiert und beinhaltet die zuvor genannten Komponenten. Die Software ist auf dem Prinzip einer Finite state machine mit 3 Zustände aufgebaut. Beim Start befindet man sich im Ready-State.

Der Zustand wartet ab, bis ein Benutzer sich über eine RFID Karte anmeldet. Wird die UID erkannt und die Authentifizierung ist erfolgreich ändert sich der Zustand in den Account-State.

Beim Versuch eines erneuten einlesen einer RFID Karte wird überprüft ob die aktuell ausgelesen UID gleich der vom eingeloggtten Benutzer ist. Ist dies der Fall, so wird der Benutzer wieder abgemeldet und der Zustand kehrt zurück zu Ready-State. Unterscheiden sich die UIDs wird auf dem LCD-Display eine Meldung ausgegeben das die Ladesäule aktuell belegt ist. Zusätzlich wird in den Zustand Ready-State gewechselt, wenn innerhalb 2 Minuten sich der Zustand Account-State sich nicht ändert. Schließt man das zu ladende Fahrzeug an ändert sich der EV-Status auf C oder D. Dieser Wiederrum führt zu einem Wechsel des Zustandes auf Load-State.

Im Load-State wird zu Beginn das Freigabebit des Ladevorgangs gesetzt, damit der Ladevorgang gestartet werden kann. Um den Ladevorgang zu beenden, gibt es zwei Möglichkeiten. Der Ladevorgang kann entweder durch das erneute Einlesen der RFID Karte beendet werden, wenn diese vom selben Benutzer stammt oder die zweite Möglichkeit besteht darin das Ladekabel abzuziehen. Erkannt wird dies durch den EV-Status da er sich auf A oder B ändert. Das Blockieren des Ladevorgangs geschieht dann bei beiden Möglichkeiten über das Löschen des Freigabebits des EV Charge Control.



Grafik 4.1.1

Beim Wechsel des Zustandes von Load-State auf Ready-State werden zwei Dateien erzeugt eine CSV-Datei und eine Javascript-Datei. Die benötigten Informationen werden in der Liste `Verbrauchsobjekt` gespeichert. Diese besteht aus 4 Elementen der UID des angemeldeten Benutzers, der gesamte Verbrauch des Ladevorgangs, Datum und Uhrzeit des Starts des Ladevorgangs und Datum und Uhrzeit des Endes des Ladevorgangs. Beim CSV-Export wird eine Datei „savefile.csv“ erzeugt und mithilfe der Liste `Verbrauchsobjekt` befüllt, falls diese Datei schon vorhanden ist, wird der Text angehängt. Die JavaScript-Datei „savefile.js“ wird zusätzlich zur CSV Datei erzeugt, um das Auslesen der Daten über den Webserver zu ermöglichen. Bei der Erzeugung dieser Datei wird der Inhalt immer wieder überschrieben. Um mehrere Ladevorgänge festzuhalten, wird daher eine zusätzliche Liste `Verbrauchsspeicher` erzeugt und dieser werden immer neue `Verbrauchsobjekte` angehängt. Der Nachteil bei der Erzeugung der JavaScript-Datei ist, dass bei Stromausfall die Daten in Verbrauchsspeicher verloren gehen und beim Neustart die Datei „savefile.js“ überschrieben wird ohne Berücksichtigung des alten Inhalts. Wieso eine zusätzliche Datei erzeugt wird und nicht direkt aus „savefile.csv“ gelesen wird ist im späteren Abschnitt ausgeführt.

4.3 Webserver

Zur übersichtlichen Darstellung des Verwendungsverlaufs der Ladesäule wurde auf dem Raspberry Pi ein Webserver installiert. Dafür wurde die Webserver Software NGINX verwendet. Für die Installation sind einige Befehle auszuführen. Für die Installation des nginx package muss `> sudo apt-get install nginx` ausgeführt werden. Anschließend zum Starten des Servers `> sudo /etc/init.d/nginx.start`. Beim Aufruf des Webserver wird die Datei `index.html` dargestellt, die unter `/var/www/html` zu finden ist.

Die HTML-Seite beinhaltet eine Tabelle, deren Aussehen durch den sich zwischen den `<style>...</style>` CSS Code bestimmt wird. Der Tabellenkopf ist statisch programmiert d.h. dieser ist immer vorhanden, selbst wenn keine zusätzlichen Daten ausgelesen werden. Um die Tabelle dynamisch zu erweitern, wird durch `<script src="index.js"></script>` eine JavaScript-Datei eingebunden. Durch `<script type="text/javascript">window.onload = getData();</script>` wird die in der `index.js` enthaltenen Funktion `getData()` ausgeführt, sobald die HTML-Seite geladen ist. Diese Funktion bereitet die Daten auf, die sich in der Datei „`savefile.js`“ befinden. Anschließend ruft sie die Funktion `display(objekt)` so oft wie benötigt auf um weitere Zeilen an die Tabelle zu hängen.

In der durch Python erzeugten Datei `savefile.js` befindet sich ein String der mit „`Var datafile =`“ beginnt. Durch die Endung `js` und die Einbindung der Datei in `index.html` mittels `<script src="savefile.js" type="text/javascript"></script>` wird dieser String als Variable `datafile` interpretiert. Die Verwendung dieser Methode wurde benutzt um Die Fehlermeldung: "Cross origin requests are only supported for protocol schemes: http, data, chrome, chrome-extension, https, chrome-extension-resource." zu vermeiden. Diese Fehlermeldung tritt beim Versuch `savefile.csv` mittels Read Funktionen auszulesen auf.

5. Fazit

Wenn man jetzt die anfänglichen Ziele nach der MoSCoW-Priorisierung mit den erfüllten vergleicht, lässt sich ein Fazit ziehen. Der erste und wichtigste Punkt war es die Funktion herzustellen. Die vorhandenen Fehler in der Verdrahtung konnten relativ früh eliminiert werden, die Funktion allerdings erst mit Fertigstellung eines weiteren Must-Punktes, dem Emulator, getestet werden. Ab diesem Zeitpunkt konnten die weiteren Funktionen gestartet und getestet werden. So wurde der RFID- Kartenleser und die dazugeschriebene Software getestet und in die Ladebox integriert. Die Montage des Displays zur Anzeige der Ladeinformationen stellte uns vor Schwierigkeiten, da sowohl die Berührungssicherheit, als auch die ästhetische Platzierung beachtet werden musste. Letztlich wurde aber eine Lösung gefunden. Die größte Hürde in diesem Projekt stellte aber die Schnittstelle zum Zähler dar. Da zu diesem Zeitpunkt das Projekt schon dem Ende zuging und trotz zahlreicher Versuche das Problem nicht gelöst werden konnte drohte dieser Must-Punkt nicht erledigt zu werden. Die Ursache wurde schließlich im Defekt des Gerätes gefunden und mit einer Last-Minute Bestellung konnte das Problem gelöst werden.

Nicht umgesetzt werden konnten die beiden Punkte mit der Could-Priorität. Auf der einen Seite die Analyse der Pulsweitenmodulation im Emulator wurde aus Zeitmangel und zu geringer Relevanz nicht bis zur Vorrüstung durchgeführt. Auf der anderen Seite die Ladeleistungsbegrenzung bei mehreren Ladeboxen wurde aus Zeitgründen nicht durchgeführt. Alles in allem wurden alle Must-Punkte trotz Schwierigkeiten umgesetzt und das Projekt rechtzeitig fertiggestellt.